

IKB42603 Cloud Computing Security Essentials

Practical Assessment - Question Paper Part 1

Hands-on technical assessment · Kubernetes · Secrets · Multi-Tenancy

Student name	Nur Shafiqah Binti Ab Rahim (L03)
Student ID	52215124832
Question(s) attempted	Question 1, 2, 3, 4
Date and session	11 September 2026

Declaration

I declare that the configuration and the evidence submitted with this paper are entirely my own work, produced by me in this session, and that I have not copied, shared or received terminal output from any other candidate.

Signature	<i>piqa</i>
Date	11 September 2026

Evidence Index

The table below identifies the screenshots that should receive the strongest attention during marking. The remaining screenshots are supporting records showing how each configuration was created or verified.

Question	Key evidence	What it proves	Status
Q1	Q1 Evidence 1 (Screenshot 10)	finance get allowed; HR get denied	COMPLETE
Q1	Q1 Evidence 2 (Screenshot 11)	list/delete denied; finance named get allowed; HR named get denied	COMPLETE
Q2	Evidence 1 (Screenshot 19)	ResourceQuota ceilings and current usage	COMPLETE
Q2	Evidence 2 (Screenshots 20A–20B)	5-pod ceiling and aggregate CPU ceiling enforced with 'exceeded quota' rejection	COMPLETE
Q3	Evidence 1 (Screenshot 27)	Public bucket policy + anonymous HTTP 200 + confidential record exposure	COMPLETE
Q3	Evidence 2 (Screenshot 28)	All four public-access-block settings true + least-privilege internal policy	COMPLETE
Q3	Evidence 3 (Screenshot 29)	Public policy reapplied and anonymous read still succeeds in LocalStack	COMPLETE
Q3	Evidence 4 (Screenshot 33)	Normal GET fails; version-specific GET succeeds	COMPLETE
Q3	Evidence 5 (Screenshots 34–37)	Version before/after deletion; delete marker remains; target version removed	COMPLETE
Q4	Evidence 1 (Screenshot 38)	KMS KeyId + encrypted archive + wrapped data key	COMPLETE
Q4	Evidence 2 (Screenshot 40)	Round-trip decryption produces MATCH	COMPLETE
Q4	Evidence 3 (Screenshot 43)	Pending deletion + KMSInvalidStateException on unwrap	COMPLETE

Question 1 - Kubernetes Secrets and RBAC

The screenshots below are arranged from environment setup to namespace/Secret/ServiceAccount creation, RBAC configuration, and the two required authorization checks. The two authorization screenshots are the key evidence.

Q1 Setup and Supporting Screenshots

Screenshot 1: Initial Kubernetes context and cluster connectivity

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> kubectl config current-context
kind-ccse-lab2
PS C:\WINDOWS\system32> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:55998
CoreDNS is running at https://127.0.0.1:55998/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 21:39:42
```

Explanation: Confirms the working Kubernetes context (kind-ccse-lab2), cluster connectivity, and the student identity/timestamp line.

Screenshot 2: Create finance and hr namespaces

```
PS C:\WINDOWS\system32> kubectl create namespace finance
namespace/finance created
PS C:\WINDOWS\system32> kubectl create namespace hr
namespace/hr created
PS C:\WINDOWS\system32> kubectl get namespaces finance hr
NAME          STATUS    AGE
finance       Active    50s
hr            Active    39s
PS C:\WINDOWS\system32> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 21:46:24
```

Explanation: Shows creation of the two tenant namespaces and confirms both are Active.

Screenshot 3: Create finance-db Secret

```
PS C:\WINDOWS\system32> kubectl create secret generic finance-db --from-literal=DB_PASSWORD=DB_PASSWORD_FINANC
secret/finance-db created
```

Explanation: Shows creation of the finance database Secret using the required key/value setup. The Secret value is not repeated in the report text.

Screenshot 4: Create hr-db Secret

```
secret/finance-db created
PS C:\WINDOWS\system32> kubectl create secret generic hr-db --from-literal=D
secret/hr-db created
```

Explanation: Shows creation of the HR database Secret.

Screenshot 5: Create finance-app ServiceAccount

```
PS C:\WINDOWS\system32> kubectl create serviceaccount finance-app -n finance
serviceaccount/finance-app created
PS C:\WINDOWS\system32> kubectl get serviceaccount finance-app -n finance
NAME          AGE
finance-app   10s
```

Explanation: Shows creation and confirmation of the finance-app ServiceAccount in the finance namespace.

Screenshot 6: Initial Role creation attempt from protected directory

```
serviceaccount/finance-app created
PS C:\WINDOWS\system32> kubectl get serviceaccount finance-app -n finance
NAME          AGE
finance-app   10s
PS C:\WINDOWS\system32> @"
>> apiVersion: rbac.authorization.k8s.io/v1
>> kind: Role
>> metadata:
>>   name: finance-secret-reader
>>   namespace: finance
>> rules:
>> - apiGroups: [""]
>>   resources: ["secrets"]
>>   verbs: ["get"]
>> @" | Set-Content finance-secret-role.yaml
Set-Content : Access to the path
'C:\WINDOWS\system32\finance-secret-role.yaml' is denied.
At line:11 char:6
+ @" | Set-Content finance-secret-role.yaml
+ ~~~~~
+ CategoryInfo          : PermissionDenied: (C:\WINDOWS\syst...ecret-r
ole.yaml:String) [Set-Content], UnauthorizedAccessException
+ FullyQualifiedErrorId : GetContentWriterUnauthorizedAccessError,Micr
osoft.PowerShell.Commands.SetContentCommand
```

Explanation: Shows the PowerShell permission error when attempting to save the Role YAML under C:\Windows\System32. This is a troubleshooting/supporting screenshot, not assessment evidence.

Screenshot 7: Move to Documents working directory

```
PS C:\WINDOWS\system32> cd $HOME\Documents
PS C:\Users\shafi\Documents> pwd

Path
----
C:\Users\shafi\Documents
```

Explanation: Shows the working directory being changed to the user Documents folder so Kubernetes YAML files can be created successfully.

Screenshot 8: Create and apply RoleBinding

```
PS C:\Users\shafi\Documents> @"
>> apiVersion: rbac.authorization.k8s.io/v1
>> kind: RoleBinding
>> metadata:
>>   name: finance-secret-reader-binding
>>   namespace: finance
>> subjects:
>> - kind: ServiceAccount
>>   name: finance-app
>>   namespace: finance
>> roleRef:
>>   kind: Role
>>   name: finance-secret-reader
>>   apiGroup: rbac.authorization.k8s.io
>> "@ | Set-Content finance-secret-rolebinding.yaml
PS C:\Users\shafi\Documents> kubectl apply -f finance-secret-rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/finance-secret-reader-binding created
```

Explanation: Shows the finance ServiceAccount being bound to the finance-secret-reader Role.

Screenshot 9: Create and apply finance-secret-reader Role

```

PS C:\Users\shafi\Documents> @"
>> apiVersion: rbac.authorization.k8s.io/v1
>> kind: Role
>> metadata:
>>   name: finance-secret-reader
>>   namespace: finance
>> rules:
>> - apiGroups: [""]
>>   resources: ["secrets"]
>>   verbs: ["get"]
>> "@ | Set-Content finance-secret-role.yaml
PS C:\Users\shafi\Documents> kubectl apply -f finance-secret-role.yaml
role.rbac.authorization.k8s.io/finance-secret-reader created

```

Explanation: Shows the Role granting only get access to Secrets in the finance namespace.

Q1 Required Evidence**Screenshot 10: Q1 evidence 1 – Secret get authorization**

```

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 21:55:56
PS C:\Users\shafi\Documents> kubectl auth can-i get secrets --as=system:serviceaccount:finance:finance-app -n finance
yes
PS C:\Users\shafi\Documents> kubectl auth can-i get secrets --as=system:serviceaccount:finance:finance-app -n hr
no

```

Explanation: The finance-app ServiceAccount can get Secrets in finance (yes) but cannot get Secrets in hr (no).

This demonstrates namespace-scoped access separation.

Screenshot 11: Q1 evidence 2 – Minimum permission checks

```
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:00:48
PS C:\Users\shafi\Documents> kubectl auth can-i list secrets --as=system:serviceaccount:finance:finance-app -n finance
no
PS C:\Users\shafi\Documents> kubectl auth can-i delete secrets --as=system:serviceaccount:finance:finance-app -n finance
no
PS C:\Users\shafi\Documents> kubectl auth can-i get secret/finance-db --as=system:serviceaccount:finance:finance-app -n finance
yes
PS C:\Users\shafi\Documents> kubectl auth can-i get secret/hr-db --as=system:serviceaccount:finance:finance-app -n hr
no
```

Explanation: The ServiceAccount cannot list or delete Secrets in finance, can get the named finance-db Secret, and cannot get the named hr-db Secret. This demonstrates least privilege and tenant isolation.

Q1 Explanation / Analysis

The RBAC design uses a namespace-scoped Role and RoleBinding. The Role grants only the get verb on Secrets in finance, so the ServiceAccount cannot list or delete Secrets and has no equivalent permission in HR. This provides tenant isolation by limiting the identity to the required namespace and operation.

Question 2 - Kubernetes ResourceQuota

The screenshots are ordered from quota creation and verification, through Pod-count enforcement, to a separate aggregate CPU enforcement test. Both enforcement cases are retained so all ResourceQuota requirements are demonstrated.

Q2 Setup and Supporting Screenshots

Screenshot 12: Create student-app ResourceQuota YAML and apply it

```
PS C:\Users\shafi\Documents> @
>> apiVersion: v1
>> kind: ResourceQuota
>> metadata:
>>   name: student-app-quota
>>   namespace: student-app
>> spec:
>>   hard:
>>     pods: "5"
>>     requests.cpu: "1"
>>     requests.memory: "512Mi"
>> "@ | Set-Content student-app-quota.yaml
PS C:\Users\shafi\Documents> kubectl apply -f student-app-quota.yaml
resourcequota/student-app-quota created
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:03:58
```

Explanation: Shows the quota definition: pods=5, requests.cpu=1, and requests.memory=512Mi, followed by successful application.

Screenshot 14: Create student-pod-1

```
PS C:\Users\shafi\Documents> @
>> apiVersion: v1
>> kind: Pod
>> metadata:
>>   name: student-pod-1
>>   namespace: student-app
>> spec:
>>   containers:
>>   - name: student-pod-1
>>     image: nginx
>>     resources:
>>       requests:
>>         cpu: "200m"
>>         memory: "100Mi"
>> "@ | Set-Content student-pod-1.yaml
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-1.yaml
pod/student-pod-1 created
PS C:\Users\shafi\Documents> kubectl get pods -n student-app
NAME          READY   STATUS    RESTARTS   AGE
student-pod-1 1/1     Running   0           8s
```

Explanation: Shows the first Pod with CPU request 200m and memory request 100Mi.

Screenshot 15: Create student-pod-2 and student-pod-3

```
PS C:\Users\shafi\Documents> @
>> apiVersion: v1
>> kind: Pod
>> metadata:
>>   name: student-pod-2
>>   namespace: student-app
>> spec:
>>   containers:
>>   - name: student-pod-2
>>     image: nginx
>>     resources:
>>       requests:
>>         cpu: "200m"
>>         memory: "100Mi"
>> "@ | Set-Content student-pod-2.yaml
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-2.yaml
pod/student-pod-2 created
PS C:\Users\shafi\Documents> @
>> apiVersion: v1
>> kind: Pod
>> metadata:
>>   name: student-pod-3
>>   namespace: student-app
>> spec:
>>   containers:
>>   - name: student-pod-3
>>     image: nginx
>>     resources:
>>       requests:
>>         cpu: "200m"
>>         memory: "100Mi"
>> "@ | Set-Content student-pod-3.yaml
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-3.yaml
pod/student-pod-3 created
```

Explanation: Shows creation of Pods 2 and 3 with the same resource requests.

Screenshot 16: Create student-pod-4 and student-pod-5

```

PS C:\Users\shafi\Documents> @
>> apiVersion: v1
>> kind: Pod
>> metadata:
>>   name: student-pod-4
>>   namespace: student-app
>> spec:
>>   containers:
>>   - name: student-pod-4
>>     image: nginx
>>     resources:
>>       requests:
>>         cpu: "200m"
>>         memory: "100Mi"
>> "@ | Set-Content student-pod-4.yaml
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-4.yaml
pod/student-pod-4 created
PS C:\Users\shafi\Documents> @
>> apiVersion: v1
>> kind: Pod
>> metadata:
>>   name: student-pod-5
>>   namespace: student-app
>> spec:
>>   containers:
>>   - name: student-pod-5
>>     image: nginx
>>     resources:
>>       requests:
>>         cpu: "200m"
>>         memory: "100Mi"
>> "@ | Set-Content student-pod-5.yaml
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-5.yaml
pod/student-pod-5 created

```

Explanation: Shows creation of Pods 4 and 5 with the same resource requests.

Screenshot 17: Five Pods running

```

PS C:\Users\shafi\Documents> kubectl get pods -n student-app

```

NAME	READY	STATUS	RESTARTS	AGE
student-pod-1	1/1	Running	0	113s
student-pod-2	1/1	Running	0	54s
student-pod-3	1/1	Running	0	33s
student-pod-4	1/1	Running	0	19s
student-pod-5	1/1	Running	0	10s

Explanation: Confirms student-pod-1 through student-pod-5 are Running.

Screenshot 18: Attempt to create student-pod-6

```

PS C:\Users\shafi\Documents> @"
>> apiVersion: v1
>> kind: Pod
>> metadata:
>>   name: student-pod-6
>>   namespace: student-app
>> spec:
>>   containers:
>>   - name: student-pod-6
>>     image: nginx
>>     resources:
>>       requests:
>>         cpu: "200m"
>>         memory: "100Mi"
>> "@ | Set-Content student-pod-6.yaml
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:10:53
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-6.yaml
Error from server (Forbidden): error when creating "student-pod-6.yaml": pod
s "student-pod-6" is forbidden: exceeded quota: student-app-quota, requested
: pods=1,requests.cpu=200m,requests.memory=100Mi, used: pods=5,requests.cpu=
1,requests.memory=500Mi, limited: pods=5,requests.cpu=1,requests.memory=512M
i

```

Explanation: Shows the over-quota creation attempt and the API rejection containing 'exceeded quota'.

Q2 Required Evidence

Screenshot 19: Evidence 1 – Active ResourceQuota and current usage

```

PS C:\Users\shafi\Documents> kubectl describe resourcequota student-app-quot
a -n student-app
Name:          student-app-quota
Namespace:     student-app
Resource       Used  Hard
-----
pods           0    5
requests.cpu   0    1
requests.memory 0    512Mi

```

Explanation: Shows namespace student-app with hard ceilings of 5 Pods, 1 CPU of aggregate requests, and 512Mi of aggregate memory requests, together with current usage.

Screenshot 20A: Q2 Evidence 2 – Pod-count ceiling rejection

```

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:12:09
PS C:\Users\shafi\Documents> kubectl get pods -n student-app
NAME             READY   STATUS    RESTARTS   AGE
student-pod-1    1/1     Running   0           5m24s
student-pod-2    1/1     Running   0           4m25s
student-pod-3    1/1     Running   0           4m4s
student-pod-4    1/1     Running   0           3m50s
student-pod-5    1/1     Running   0           3m41s
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-6.yaml
Error from server (Forbidden): error when creating "student-pod-6.yaml": pod
s "student-pod-6" is forbidden: exceeded quota: student-app-quota, requested
: pods=1,requests.cpu=200m,requests.memory=100Mi, used: pods=5,requests.cpu=
1,requests.memory=500Mi, limited: pods=5,requests.cpu=1,requests.memory=512M
i

```

Explanation: Shows five running Pods followed by the rejected sixth Pod creation. The error explicitly contains “exceeded quota”, demonstrating enforcement of the Pod-count ceiling.

Screenshot 20B: Q2 Evidence 2 – Aggregate CPU request ceiling rejection

```

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/12/2026 04:01:21
PS C:\Users\shafi\Documents> kubectl get pods -n student-app
NAME             READY   STATUS    RESTARTS   AGE
student-pod-1    1/1     Running   0           2m52s
student-pod-2    1/1     Running   0           2m52s
student-pod-3    1/1     Running   0           2m51s
student-pod-4    1/1     Running   0           2m41s
PS C:\Users\shafi\Documents> kubectl describe resourcequota student-ap
p-quota -n student-app
Name:             student-app-quota
Namespace:        student-app
Resource          Used    Hard
-----
pods              4      5
requests.cpu      800m   1
requests.memory   400Mi  512Mi
PS C:\Users\shafi\Documents> kubectl apply -f student-pod-cpu.yaml
Error from server (Forbidden): error when creating "student-pod-cpu.ya
ml": pods "student-pod-cpu" is forbidden: exceeded quota: student-app-
quota, requested: requests.cpu=300m, used: requests.cpu=800m, limited:
requests.cpu=1

```

Explanation: Shows the preceding state of 4 Running Pods with quota usage at 800m CPU and 400Mi memory, followed by a fifth Pod request for 300m CPU being rejected. The API reports “exceeded quota” and explicitly

identifies requests.cpu=300m, used=800m, and limited=1 CPU. The Pod count would have been 5/5 and memory 500Mi/512Mi, so the rejection demonstrates the aggregate CPU ceiling rather than the Pod-count ceiling.

Coverage note: Q2 Evidence 2 is presented as Screenshots 20A and 20B, covering both required enforcement cases: the Pod-count ceiling rejection and a separate aggregate CPU request ceiling rejection.

Q2 Explanation / Analysis

The noisy-neighbour problem occurs when one workload consumes excessive shared cluster resources and affects other workloads. The ResourceQuota limits the total number of Pods and aggregate CPU and memory requests in student-app, preventing the namespace from exceeding its allocation and helping maintain fair resource isolation in a shared multi-tenant cluster.

Question 3 - S3 Object Storage, Public Exposure and Data

Remanence

The screenshots are arranged in the logical workflow: bucket creation, object upload/classification, insecure public policy, public-access blocking and least-privilege replacement, the LocalStack enforcement observation, versioning, and version deletion.

Q3 Setup and Supporting Screenshots

Screenshot 21: Create patient-records bucket

```
PS C:\Users\shafi\Documents> $BUCKET="52215124832-patient-records"  
PS C:\Users\shafi\Documents> aws $EP s3 mb s3://$BUCKET  
make_bucket: 52215124832-patient-records
```

Explanation: Shows the student-specific bucket 52215124832-patient-records being created.

Screenshot 22: Upload public, internal and confidential objects

```
PS C:\Users\shafi\Documents> aws $EP s3 cp notice.txt s3://$BUCKET/public/no  
tice.txt  
upload: .\notice.txt to s3://52215124832-patient-records/public/notice.txt  
PS C:\Users\shafi\Documents> aws $EP s3 cp roster.txt s3://$BUCKET/internal/  
roster.txt  
upload: .\roster.txt to s3://52215124832-patient-records/internal/roster.txt  
PS C:\Users\shafi\Documents> aws $EP s3 cp record.txt s3://$BUCKET/confident  
ial/record.txt  
upload: .\record.txt to s3://52215124832-patient-records/confidential/record  
.txt
```

Explanation: Shows the three objects being uploaded to their respective prefixes.

Screenshot 23: Apply object classification tags

```

PS C:\Users\shafi\Documents> aws $EP s3api put-object-tagging --bucket $BUCKET --key public/notice.txt --tagging "TagSet=[{Key=Classification,Value=Public}]"
PS C:\Users\shafi\Documents> aws $EP s3api put-object-tagging --bucket $BUCKET --key internal/roster.txt --tagging "TagSet=[{Key=Classification,Value=Internal}]"
PS C:\Users\shafi\Documents> aws $EP s3api put-object-tagging --bucket $BUCKET --key confidential/record.txt --tagging "TagSet=[{Key=Classification,Value=Confidential}]"
PS C:\Users\shafi\Documents> aws $EP s3api get-object-tagging --bucket $BUCKET --key confidential/record.txt
{
  "TagSet": [
    {
      "Key": "Classification",
      "Value": "Confidential"
    }
  ]
}

```

Explanation: Shows Classification tags being applied and the confidential object tag being verified as Confidential.

Screenshot 24: Create and apply insecure public bucket policy

```

PS C:\Users\shafi\Documents> @"
>> {
>>   "Version": "2012-10-17",
>>   "Statement": [
>>     {
>>       "Effect": "Allow",
>>       "Principal": "*",
>>       "Action": "s3:GetObject",
>>       "Resource": "arn:aws:s3:::$BUCKET/*"
>>     }
>>   ]
>> }
>> @" | Set-Content public-policy.json
PS C:\Users\shafi\Documents> aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://public-policy.json
PS C:\Users\shafi\Documents> aws $EP s3api get-bucket-policy --bucket $BUCKET
{
  "Policy": "{\n  \"Version\": \"2012-10-17\", \n  \"Statement\": [\n    {\n      \"Effect\": \"Allow\", \n      \"Principal\": \"*\", \n      \"Action\": \"s3:GetObject\", \n      \"Resource\": \"arn:aws:s3:::52215124832-patient-records/*\" \n    } \n  ] \n}"
}

```

Explanation: Shows Principal '*' with s3:GetObject on all objects in the bucket.

Screenshot 25: Configure S3 Block Public Access

```

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:27:29
PS C:\Users\shafi\Documents> aws $EP s3api get-public-access-block --b
ucket $BUCKET
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}

```

Explanation: Shows the four public-access-block settings being enabled.

Screenshot 26: Supporting public-policy cleanup / configuration

```

PS C:\Users\shafi\Documents> Get-Content leaked.txt
Patient: Ahmad, Diagnosis: confidential
PS C:\Users\shafi\Documents> aws $EP s3api delete-bucket-policy --buck
et $BUCKET
PS C:\Users\shafi\Documents> aws $EP s3api get-bucket-policy --bucket
$BUCKET

aws: [ERROR]: An error occurred (NoSuchBucketPolicy) when calling the
GetBucketPolicy operation: The bucket policy does not exist
PS C:\Users\shafi\Documents> aws $EP s3api put-public-access-block --b
ucket $BUCKET --public-access-block-configuration BlockPublicAcls=true
,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=tr
ue

```

Explanation: Shows the public policy removal and the subsequent public-access-block configuration workflow.

Q3 Required Evidence – Public Exposure

Screenshot 27: Q3 Evidence 1 – Public policy and anonymous data exposure

```
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:25:16
PS C:\Users\shafi\Documents> aws $EP s3api get-bucket-policy --bucket
$BUCKET
{
  "Policy": "{\n  \"Version\": \"2012-10-17\", \n  \"Statement\": [\n
    {\n    \"Effect\": \"Allow\", \n    \"Principal\": \"*\", \n
    \"Action\": \"s3:GetObject\", \n    \"Resource\": \"arn:aws:s3:::52
215124832-patient-records/*\" \n    } \n  ] \n}"
}

PS C:\Users\shafi\Documents> curl.exe -s -o leaked.txt -w "HTTP %{http
_code}" "http://localhost:4566/$BUCKET/confidential/record.txt"
HTTP 200
PS C:\Users\shafi\Documents> Get-Content leaked.txt
Patient: Ahmad, Diagnosis: confidential
```

Explanation: Shows the public bucket policy with Principal ‘*’, an anonymous curl request returning HTTP 200, and the confidential patient record being returned. The offending access-control mechanism is the S3 bucket policy.

Screenshot 28: Q3 Evidence 2 – Public-access block and least-privilege policy

```
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:34:45
PS C:\Users\shafi\Documents> aws $EP s3api get-public-access-block --bucket $BUCKET
{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}

PS C:\Users\shafi\Documents> aws $EP s3api get-bucket-policy --bucket $BUCKET
{
  "Policy": "{\n  \"Version\": \"2012-10-17\", \n  \"Statement\": [\n    {\n      \"Effect\": \"Allow\", \n      \"Principal\": {\n        \"AWS\": \"arn:aws:iam::000000000000:root\" \n      }, \n      \"Action\": \"s3:GetObject\", \n      \"Resource\": \"arn:aws:s3:::52215124832-patient-re
cords/internal/*\" \n    } \n  ] \n}"
}
```

Explanation: Shows all four Block Public Access settings as true and the replacement policy restricted to the internal/ prefix and the account principal.

Screenshot 29: Q3 Evidence 3 – Reapplied public policy in LocalStack

```

PS C:\Users\shafi\Documents> aws $EP s3api put-bucket-policy --bucket
$BUCKET --policy file:///public-policy.json
PS C:\Users\shafi\Documents> curl.exe -s -o leaked-after-block.txt -w
"HTTP %{http_code}`n" "http://localhost:4566/$BUCKET/confidential/reco
rd.txt"
HTTP 200
PS C:\Users\shafi\Documents> Get-Content leaked-after-block.txt
Patient: Ahmad, Diagnosis: confidential

```

Explanation: Shows the identical public policy being reapplied and the anonymous request still returning HTTP 200 in LocalStack. This is an important environment-specific observation.

Q3 Public Exposure Analysis

The offending character is ‘*’ in the Principal field because it represents everyone and permits anonymous access. The misused mechanism is an S3 bucket policy. In real AWS, BlockPublicPolicy is a preventative guardrail that rejects a public bucket policy before it becomes effective; a detective control would only identify the exposure after it occurred. The screenshots also document the observed LocalStack behaviour, rather than assuming LocalStack enforces the same way as real AWS.

Q3 Versioning and Data Remanence

Screenshot 30: Enable and verify bucket versioning

```

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:41:12
PS C:\Users\shafi\Documents> aws $EP s3api put-bucket-versioning --buc
ket $BUCKET --versioning-configuration Status=Enabled
PS C:\Users\shafi\Documents> aws $EP s3api get-bucket-versioning --buc
ket $BUCKET
{
  "Status": "Enabled"
}

```

Explanation: Shows versioning being enabled for the bucket and confirmed as Enabled.

Screenshot 31: List object versions – supporting before state

```

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:47:57
PS C:\Users\shafi\Documents> aws $EP s3api list-object-versions --bucket $BUCKET --prefix confidential/record.txt
{
  "Versions": [
    {
      "ETag": "\"292e4715e178beac2b7175474f7547b8\"",
      "ChecksumAlgorithm": [
        "CRC64NVME"
      ],
      "ChecksumType": "FULL_OBJECT",
      "Size": 41,
      "StorageClass": "STANDARD",
      "Key": "confidential/record.txt",
      "VersionId": "AaCQ7irhXRjHsZrjYIPAbWiH_r0BR059",
      "IsLatest": true,
      "LastModified": "2026-09-11T14:47:47+00:00",
      "Owner": {
        "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
      }
    },
    {
      "ETag": "\"292e4715e178beac2b7175474f7547b8\"",
      "ChecksumAlgorithm": [
        "CRC64NVME"
      ],
      "ChecksumType": "FULL_OBJECT",
      "Size": 41,
      "StorageClass": "STANDARD",
      "Key": "confidential/record.txt",
      "VersionId": "AaCQ7irg3orYqw54hn3P0PSYiZ0ZgzIn",
      "IsLatest": false,
      "LastModified": "2026-09-11T14:45:22+00:00",
      "Owner": {
        "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
      }
    }
  ],
  "RequestCharged": null,
  "Prefix": "confidential/record.txt"
}

```

Explanation: Shows multiple versions of confidential/record.txt, including non-null VersionIds.

Screenshot 32: Detailed version listing

```

      "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
    },
    {
      "ETag": "\"292e4715e178beac2b7175474f7547b8\"",
      "ChecksumAlgorithm": [
        "CRC64NVME"
      ],
      "ChecksumType": "FULL_OBJECT",
      "Size": 41,
      "StorageClass": "STANDARD",
      "Key": "confidential/record.txt",
      "VersionId": "null",
      "IsLatest": false,
      "LastModified": "2026-09-11T14:17:36+00:00",
      "Owner": {
        "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
      }
    }
  ],
  "RequestCharged": null,
  "Prefix": "confidential/record.txt"
}

```

Explanation: Provides additional version-listing output, including the target version and delete-marker information.

Q3 Required Evidence – Versioned Deletion

Screenshot 33: Q3 Evidence 4 – Failed ordinary GET and successful versioned GET

```
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 22:58:42
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> aws $EP s3api get-object --bucket $BUCKET
--key confidential/record.txt normal-get.txt

aws: [ERROR]: An error occurred (NoSuchKey) when calling the GetObject
operation: The specified key does not exist.
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> aws $EP s3api get-object --bucket $BUCKET
--key confidential/record.txt --version-id AaCQ7irg3orYqw54hn3P0PSYiZ
0ZgzIn recovered-old.txt
{
  "AcceptRanges": "bytes",
  "LastModified": "2026-09-11T14:45:22+00:00",
  "ContentLength": 41,
  "ETag": "\"292e4715e178beac2b7175474f7547b8\"",
  "ChecksumCRC64NVME": "UvjaeTFLPL4=",
  "ChecksumType": "FULL_OBJECT",
  "VersionId": "AaCQ7irg3orYqw54hn3P0PSYiZ0ZgzIn",
  "ContentType": "text/plain",
  "ServerSideEncryption": "AES256",
  "Metadata": {}
}
```

Explanation: The normal GET fails with `NoSuchKey` because a delete marker is current, while a version-specific GET using `VersionId` succeeds and returns object metadata. The `VersionId` is visible in the command and output.

Screenshot 34: Q3 Evidence 5 – Version listing before permanent deletion (part 1)

```

PS C:\Users\shafi\Documents> aws $EP s3api list-object-versions --bucket $BUCKET --prefix confidential/record.txt
{
  "Versions": [
    {
      "ETag": "\"292e4715e178beac2b7175474f7547b8\"",
      "ChecksumAlgorithm": [
        "CRC64NVME"
      ],
      "ChecksumType": "FULL_OBJECT",
      "Size": 41,
      "StorageClass": "STANDARD",
      "Key": "confidential/record.txt",
      "VersionId": "AaCQ7irhXRjHsZrjYIPAbWiH_r0BR059",
      "IsLatest": false,
      "LastModified": "2026-09-11T14:47:47+00:00",
      "Owner": {
        "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
      }
    },
    {
      "ETag": "\"292e4715e178beac2b7175474f7547b8\"",
      "ChecksumAlgorithm": [
        "CRC64NVME"
      ],
      "ChecksumType": "FULL_OBJECT",
      "Size": 41,
      "StorageClass": "STANDARD",
      "Key": "confidential/record.txt",
      "VersionId": "null",
      "IsLatest": false,
      "LastModified": "2026-09-11T14:17:36+00:00",
      "Owner": {
        "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
      }
    }
  ],
  "DeleteMarkers": [

```

Explanation: Shows the target version AaCQ7irg3orYqw54hn3P0PSYiZ0ZgZln still present before permanent deletion, alongside the other version(s).

Screenshot 35: Q3 Evidence 5 – Version listing before permanent deletion (part 2)

```

    },
    "DeleteMarkers": [
      {
        "Owner": {
          "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
        },
        "Key": "confidential/record.txt",
        "VersionId": "AaCQ7iriJzX3BfZiAiouexxolSRhLla5w",
        "IsLatest": true,
        "LastModified": "2026-09-11T14:50:54+00:00"
      },
      {
        "Owner": {
          "ID": "75aa57f09aa0c8caeab4f8c24e99d10f8e7faeebf76c078efc7c6caea54ba06a"
        },
        "Key": "confidential/record.txt",
        "VersionId": "AaCQ7iri4rTJbJ9s_TNRCdZRqJvbkK2.",
        "IsLatest": false,
        "LastModified": "2026-09-11T14:50:45+00:00"
      }
    ],
    "RequestCharged": null,
    "Prefix": "confidential/record.txt"
  }
}

```

Explanation: Shows the delete-marker section, including the latest delete marker. Together with Screenshot 33 this establishes the before state.

Screenshot 36: Q3 Evidence 5 – Permanent deletion command

```

PS C:\Users\shafi\Documents> $AES = [Security.Cryptography.Aes]::Create()
PS C:\Users\shafi\Documents> $AES.KeySize = 256
PS C:\Users\shafi\Documents> $AES.Mode = [Security.Cryptography.CipherMode]::CBC
PS C:\Users\shafi\Documents> $AES.Padding = [Security.Cryptography.PaddingMode]::PKCS7
PS C:\Users\shafi\Documents> $AES.Key = $PLAINTEXT_KEY
PS C:\Users\shafi\Documents> $AES.IV = $IV
PS C:\Users\shafi\Documents> $InputData = [IO.File]::ReadAllBytes("$PWD\record.txt")
PS C:\Users\shafi\Documents> $Encryptor = $AES.CreateEncryptor()
PS C:\Users\shafi\Documents> $EncryptedData = $Encryptor.TransformFinalBlock($InputData, 0, $InputData.Length)
PS C:\Users\shafi\Documents> [IO.File]::WriteAllBytes("$PWD\record.env.enc", $IV + $EncryptedData)
PS C:\Users\shafi\Documents> $Encryptor.Dispose()
PS C:\Users\shafi\Documents> $AES.Dispose()
PS C:\Users\shafi\Documents> $IV = New-Object byte[] 16
PS C:\Users\shafi\Documents> $RNG = [Security.Cryptography.RandomNumberGenerator]::Create()
PS C:\Users\shafi\Documents> $RNG.GetBytes($IV)
PS C:\Users\shafi\Documents> $RNG.Dispose()
PS C:\Users\shafi\Documents> $AES = [Security.Cryptography.Aes]::Create()
PS C:\Users\shafi\Documents> $AES.KeySize = 256
PS C:\Users\shafi\Documents> $AES.Mode = [Security.Cryptography.CipherMode]::CBC
PS C:\Users\shafi\Documents> $AES.Padding = [Security.Cryptography.PaddingMode]::PKCS7
PS C:\Users\shafi\Documents> $AES.Key = $PLAINTEXT_KEY
PS C:\Users\shafi\Documents> $AES.IV = $IV
PS C:\Users\shafi\Documents> $InputData = [IO.File]::ReadAllBytes("$PWD\record.txt")
PS C:\Users\shafi\Documents> $Encryptor = $AES.CreateEncryptor()
PS C:\Users\shafi\Documents> $EncryptedData = $Encryptor.TransformFinalBlock($InputData, 0, $InputData.Length)
PS C:\Users\shafi\Documents> [IO.File]::WriteAllBytes("$PWD\record.env.enc", $IV + $EncryptedData)
PS C:\Users\shafi\Documents> $Encryptor.Dispose()
PS C:\Users\shafi\Documents> $AES.Dispose()

```

Explanation: Shows the specific target VersionId being permanently deleted. The returned VersionId confirms the requested version deletion operation.

Screenshot 37: Q3 Evidence 5 – Version listing after permanent deletion

```

PS C:\Users\shafi\Documents> Get-Item .\record.env.enc, .\datakey.enc | Select-Object Name,Length

```

Name	Length
record.env.enc	64
datakey.enc	116

Explanation: Shows the target VersionId is absent after deletion, while another version remains and the delete marker is still visible.

Q3 Versioning Analysis

An ordinary delete on a versioned bucket creates a delete marker instead of removing existing object versions. The object therefore becomes inaccessible through a normal GET, but previous versions remain recoverable when a VersionId is supplied. Permanent deletion requires deleting the specific version containing the data. This is important for data-erasure requests because an ordinary delete does not necessarily remove historical copies.

Question 4 - KMS Envelope Encryption and Cryptographic Erasure

The screenshots document creation of the KMS master key, generation and wrapping of an AES-256 data key, local AES encryption, successful round-trip decryption, and the effect of disabling/scheduling deletion of the KMS key.

Q4 Supporting / Process Screenshots

Screenshot 38: Q4 Evidence 1 – Encryption artifacts

```
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 23:11:22
PS C:\Users\shafi\Documents> echo "KeyId: $KEY_ID"
KeyId: 4f7d81d6-92f1-44e2-a1ba-861efcd85b72
PS C:\Users\shafi\Documents> Get-Item .\record.env.enc, .\datakey.enc
| Select-Object Name,Length
```

Name	Length
record.env.enc	64
datakey.enc	116

Explanation: Shows the KMS KeyId together with record.env.enc and datakey.enc. The plaintext data key is not shown.

Screenshot 39: Decrypt wrapped data key and decrypt archive

```

PS C:\Users\shafi\Documents> $DEC = aws $EP kms decrypt --ciphertext-b
lob fileb://datakey.enc | ConvertFrom-Json
PS C:\Users\shafi\Documents> $RECOVERED_KEY = [Convert]::FromBase64Str
ing($DEC.Plaintext)
PS C:\Users\shafi\Documents> $ENC = [IO.File]::ReadAllBytes("$PWD\reco
rd.env.enc")
PS C:\Users\shafi\Documents> $IV2 = $ENC[0..15]
PS C:\Users\shafi\Documents> $Ciphertext2 = $ENC[16..($ENC.Length-1)]
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> $AES2 = [Security.Cryptography.Aes]::Crea
te()
PS C:\Users\shafi\Documents> $AES2.KeySize = 256
PS C:\Users\shafi\Documents> $AES2.Mode = [Security.Cryptography.Ciphe
rMode]::CBC
PS C:\Users\shafi\Documents> $AES2.Padding = [Security.Cryptography.Pa
ddingMode]::PKCS7
PS C:\Users\shafi\Documents> $AES2.Key = $RECOVERED_KEY
PS C:\Users\shafi\Documents> $AES2.IV = $IV2
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> $Decryptor = $AES2.CreateDecryptor()
PS C:\Users\shafi\Documents> $RecoveredData = $Decryptor.TransformFina
lBlock($Ciphertext2, 0, $Ciphertext2.Length)
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> [IO.File]::WriteAllBytes("$PWD\recovered-
record.txt", $RecoveredData)
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> $Decryptor.Dispose()
PS C:\Users\shafi\Documents> $AES2.Dispose()

```

Explanation: Shows the process of using KMS to unwrap the data key and AES-256/CBC/PKCS7 to decrypt the archive into a recovered record file. Plaintext is not printed.


```
PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 23:14:08
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> $OriginalHash = (Get-FileHash .\record.txt -Algorithm SHA256).Hash
PS C:\Users\shafi\Documents> $RecoveredHash = (Get-FileHash .\recovered-record.txt -Algorithm SHA256).Hash
PS C:\Users\shafi\Documents>
PS C:\Users\shafi\Documents> if ($OriginalHash -eq $RecoveredHash) {
>>     "MATCH"
>> } else {
>>     "MISMATCH"
>> }
MATCH
```

Explanation: Compares SHA-256 hashes of the original and recovered record and returns MATCH.

Screenshot 41: Disable KMS key and verify state

```
PS C:\Users\shafi\Documents> aws $EP kms disable-key --key-id $KEY_ID
PS C:\Users\shafi\Documents> aws $EP kms describe-key --key-id $KEY_ID
{
  "KeyMetadata": {
    "AWSAccountId": "000000000000",
    "KeyId": "4f7d81d6-92f1-44e2-alba-861efcd85b72",
    "Arn": "arn:aws:kms:us-east-1:000000000000:key/4f7d81d6-92f1-44e2-alba-861efcd85b72",
    "CreationDate": "2026-09-11T23:05:25.373580+08:00",
    "Enabled": false,
    "Description": "52215124832 tenant-A master key",
    "KeyUsage": "ENCRYPT_DECRYPT",
    "KeyState": "Disabled",
    "Origin": "AWS_KMS",
    "KeyManager": "CUSTOMER",
    "CustomerMasterKeySpec": "SYMMETRIC_DEFAULT",
    "KeySpec": "SYMMETRIC_DEFAULT",
    "EncryptionAlgorithms": [
      "SYMMETRIC_DEFAULT"
    ],
    "MultiRegion": false,
    "CurrentKeyMaterialId": "aeab023c38a8504184985f03a0f0bceeaeab023c38a8504184985f03a0f0bcee"
  }
}
```

Explanation: Shows the key changed to Disabled, including Enabled=false and KeyState=Disabled.

Screenshot 42: Schedule KMS key deletion

```
PS C:\Users\shafi\Documents> aws $EP kms schedule-key-deletion --key-id $KEY_ID --pending-window-in-days 7
{
  "KeyId": "4f7d81d6-92f1-44e2-a1ba-861efcd85b72",
  "DeletionDate": "2026-09-18T23:17:02.212303+08:00",
  "KeyState": "PendingDeletion",
  "PendingWindowInDays": 7
}
```

Explanation: Shows PendingDeletion with a 7-day pending window and a deletion date

Q4 Required Evidence

Screenshot 43: Q4 Evidence 3 – KMS key pending deletion and unwrap failure

```
PS C:\Users\shafi\Documents> aws $EP kms schedule-key-deletion --key-id $KEY_ID --pending-window-in-days 7
{
  "KeyId": "4f7d81d6-92f1-44e2-a1ba-861efcd85b72",
  "DeletionDate": "2026-09-18T23:17:02.212303+08:00",
  "KeyState": "PendingDeletion",
  "PendingWindowInDays": 7
}

PS C:\Users\shafi\Documents> echo "52215124832 $(Get-Date)"
52215124832 09/11/2026 23:18:23
PS C:\Users\shafi\Documents> aws $EP kms decrypt --ciphertext-blob fileb:/datakey.enc

aws: [ERROR]: An error occurred (KMSInvalidStateException) when calling the Decrypt operation: arn:aws:kms:us-east-1:000000000000:key/4f7d81d6-92f1-44e2-a1ba-861efcd85b72 is pending deletion.
```

Explanation: Shows the key in PendingDeletion with a 7-day window and a subsequent KMS decrypt attempt failing with KMSInvalidStateException. No plaintext data key is disclosed.

Q4 Explanation / Analysis

22. Explain envelope encryption in your own words, and say why only the master key needs hardware-grade protection even though the archive may be terabytes. [1 mark]

Envelope encryption uses a **data key to encrypt the actual file**, while the **KMS master key is used to protect the data key**. The master key is small and only used for the data key, so it does not need to encrypt the whole terabyte-sized archive.

23. Explain how cryptographic erasure achieves provable deletion in a shared cloud where overwriting the physical storage is impossible. [1 mark]

Cryptographic erasure works by **making the master key unavailable** so the wrapped data key cannot be decrypted anymore. This means the encrypted file cannot be recovered even though the physical storage is still there.

24. State one circumstance in which cryptographic erasure would fail to make the data unrecoverable. [1 mark]

Cryptographic erasure may fail if there is **another copy of the plaintext data or data key somewhere else**, such as a backup. Removing the master key would not affect that separate copy.

Assessment Coverage Audit

Question	Required coverage	Report status
Q1 – Secrets/RBAC	Namespaces, Secrets, finance-app ServiceAccount, minimum get-only RBAC, no HR access, can-i allowed/denied checks, list/delete denied, namespace-scope analysis	Covered
Q2 – ResourceQuota	student-app; Pods=5, CPU requests=1 CPU, memory requests=512Mi; active usage; Pod-count rejection; aggregate CPU/memory rejection; noisy-	Covered - CPU enforcement added as Screenshot 20B

	neighbour analysis	
Q3 – Object Storage	Public exposure, anonymous HTTP 200, offending Principal, Block Public Access all four flags, least-privilege internal/* policy, LocalStack outcome, versioning, delete marker, recovery and permanent version deletion, erasure analysis	Covered
Q4 – Key Management	KMS master key, AES-256 data key, encrypted archive, raw wrapped key, plaintext key cleanup, MATCH round trip, key disable/PendingDeletion, failed unwrap, three technical analyses	Covered

Final Submission Checklist

- Q1 Evidence 1 and Evidence 2 are clearly labelled and show the required authorization outcomes.
- Q2 Evidence 1 shows the ResourceQuota ceilings and current usage; Evidence 2 includes both a genuine Pod-count “exceeded quota” rejection and a separate aggregate CPU “exceeded quota” rejection.
- Q3 Evidence 1 shows public exposure; Evidence 2 shows the public-access block and least-privilege policy; Evidence 3 records the LocalStack observation; Evidence 4 and 5 demonstrate versioned data remanence and permanent version deletion.
- Q4 Evidence 1 shows encrypted artifacts without exposing the plaintext data key; Evidence 2 shows MATCH; Evidence 3 shows PendingDeletion and failed unwrap.
- Supporting/setup screenshots are retained so the marker can follow the full workflow, including troubleshooting and intermediate configuration steps.